HUNT-SAGE · THREAT HUNTING CHEAT SHEET

Splunk SPL for Hunters

Commands, stats and eval functions, tstats and data models, Enterprise Security notes, and ready-to-run hunting searches for Splunk 9.x and 10.

github.com/iamjammie/hunt-sage · iamjammie.github.io/hunt-sage · Built 2026-11-03

Anatomy

```
index=wineventlog sourcetype="XmlWinEventLog:Security" EventCode=4688 earliest=-7d@d latest=now
    NOT user="*$" TERM(powershell.exe)
| rex field=Process_Command_Line "(?<flag>-enc\S*)\s+(?<b64>\S+)"
| eval cmdlen=len(Process_Command_Line)
| stats count dc(host) as hosts values(user) as users by New_Process_Name flag
| where hosts<3
| sort - count
```

- Terms in the base search are implicitly **AND**; use `OR`, `NOT`, parentheses; `*` wildcards (leading wildcards are slow).
- `TERM(value)` matches an indexed term exactly (fast, no field extraction); `CASE(Value)` forces case-sensitive matching.
- Time: `earliest=-24h`, `-7d@d` (snap to day), `@w1` (Monday), absolute `earliest="11/01/2026:00:00:00"`; `_time` is epoch seconds, `_raw` the event text.
- Field extraction happens at search time for most sourcetypes; indexed fields (`index`, `sourcetype`, `source`, `host`) and `TERM()` are the only things that use the index.
- Subsearches `[search ...]` are limited (default 10,000 results, 60 s) and run first; prefer `stats` or `lookup` to them.
- Comments: wrap text in three backticks anywhere in a search (Splunk ignores it); the examples below put the explanation above each search instead.
- Reading the block above: base search (index, sourcetype, indexed terms, time), then `rex` extracts at search time, `eval` computes, `stats` reduces.

Core commands

Command	Does	Hunting note
`stats`	aggregate by fields	the workhorse; replaces most `transaction` / `join`
`eventstats`	add aggregates to every event	compare an event with its group (`eventstats avg(bytes) as avg_bytes by host`)
`streamstats`	running aggregates in order	intervals (`streamstats current=f last(_time) as prev by src dest`), sequences, `window=`
`tstats`	aggregate from indexed fields / data models	10-100x faster; see below
`chart`, `timechart`	pivot / time series	`timechart span=1h count by user limit=10 useother=f`
`bin` (alias `bucket`)	discretise `_time` or numbers	`bin _time span=10m` before `stats`
`top`, `rare`	frequent / infrequent values	`rare limit=20 parent_process process showperc=f`
`dedup`	first event per key	`dedup host sortby -_time` (sort inside dedup, not before)
`sort`, `head`, `tail`	order and limit	`sort 0 - count` (0 = unlimited)
`table`, `fields`	choose columns	`fields` early is cheaper than `table` (which is a transforming command)
`rename`, `eval`, `where`	shape, compute, filter	`where` compares fields (`where a>b`); `search` compares to literals
`rex`	regex extract or `mode=sed` replace	`rex field=_raw "(?<user>\w+)@"` ; `rex mode=sed "s/\d{4}-\d{4}/XXXX/g"`
`regex`	keep events matching	`regex process="(?i)\\\\users\\\\.*\\.exe"` (escape backslashes twice)
`spath`	JSON / XML path extraction	`spath input=_raw path=requestObject.spec.hostPID output=hostpid`

Command	Does	Hunting note
`lookup`, `inputlookup`, `outputlookup`	enrich from CSV/KV store, read it, write it	baselines: `outputlookup known_parents.csv` then `lookup ... OUTPUTNEW`
`transaction`	group events into sessions	slow, memory-bound; use `stats range(_time) values() by id` instead
`join`	SQL-style join	subsearch-limited and slow; use `stats values() by key` or `lookup`
`append`, `appendcols`, `multisearch`	combine result sets	`multisearch` runs streaming searches in parallel
`map`, `foreach`	run a search per result / iterate fields	`foreach *_count [eval total=coalesce(total,0)+'<<FIELD>>']`
`mvexpand`, `makemv`, `mvcombine`, `mvindex`, `mvcount`, `mvfilter`	multivalue handling	`mvexpand` after `values()` to pivot back to rows
`fillnull`, `filldown`	default values	`fillnull value="-" user`
`convert`, `strftime`, `strptime`	time formatting	`eval t=strftime(_time,"%Y-%m-%d %H:%M")`
`iplocation`, `cidrmatch`	GeoIP, subnet test	`where cidrmatch("10.0.0.0/8", src_ip)`
`tojson`, `fromjson`, `json_extract`	JSON in/out (9.x)	
`untable`, `xyseries`	reshape wide/long	after `chart` for further stats
`cluster`, `kmeans`, `anomalydetection`, `anomalies`	similarity and outliers	`cluster showcount=t t=0.8 field=process` groups near-identical command lines
`predict`, `trendline`, `autoregress`, `delta`	forecasting and change	`delta count as change` on a timechart
`fieldsummary`, `metadata`, `metasearch`	schema discovery	`metadata type=sourcetypes index=*` : what do we even have?
`datamodel`, `from datamodel:`	query a data model	`| from datamodel:"Endpoint"."Processes"`
`mstats`, `msearch`	metrics indexes	host telemetry metrics (CPU, logon rates)
`collect`, summary indexing	write results to an index	daily baselines into `index=summary`
`makeresults`, `gentimes`	synthetic rows	testing eval logic: `makeresults count=1	eval x="test"`
`rest`	Splunk REST API	`rest /services/saved/searches` : audit your own detections

Stats functions

Function	Returns
`count`, `count(field)`, `dc(field)`, `estdc(field)`	rows, non-null rows, distinct (exact / estimated)
`values(f)`, `list(f)`	distinct sorted multivalue / all in order (cap: `maxvalues`)
`earliest(f)`, `latest(f)`, `earliest_time(f)`, `latest_time(f)`	value / timestamp at the first and last event
`first(f)`, `last(f)`	by result order (not time)
`min`, `max`, `range`, `sum`, `avg`, `median`, `mode`	numeric
`perc95(f)`, `p99(f)`, `exactperc90(f)`	percentiles
`stdev`, `stdevp`, `var`, `varp`	spread (beaconing, volume anomalies)
`sumsq`, `rate(f)`, `per_second(f)`	derived

Pattern: `stats count dc(dest) as dests values(dest) as dest_list earliest(_time) as first latest(_time) as last by user` then `| eval first=strftime(first,"%F %T")`.

Eval functions for hunting

Function	Use
`len(s)`, `lower(s)`, `upper(s)`, `trim(s)`, `substr(s,1,10)`	basics

Function	Use
`split(s,"\\")`, `mvindex(mv,-1)`, `mvjoin(mv,";")`, `mvdedup(mv)`, `mvfilter(match(mv,"\.exe$"))`, `mvcount(mv)`	path and list handling; `mvindex(split(process,"\\"),-1)` = file name
`match(s,"(?i)regex")`, `like(s,"%pattern%")`, `searchmatch("term")`	tests
`replace(s,"regex","repl")`, `urldecode(s)`, `printf("%08x", n)`	transforms (base64 decoding needs an app such as URL Toolbox or CyberChef)
`md5(s)`, `sha1(s)`, `sha256(s)`	hashes for keys
`tonumber(s)`, `tostring(n,"hex")`, `round(n,2)`, `ceiling`, `floor`, `log`, `exp`, `pow`	numeric
`strftime(t,"%F %T")`, `strptime(s,"%d/%m/%Y")`, `relative_time(now(),"-7d@d")`, `now()`	time
`if(c,a,b)`, `case(c1,v1,c2,v2,true(),d)`, `coalesce(a,b)`, `nullif(a,b)`, `isnull(x)`, `isnotnull(x)`	branching and nulls
`cidrmatch("10.0.0.0/8", ip)`, `ipmask("255.255.0.0", ip)`	networks
`json_extract(j,"path")`, `json_keys(j)`, `spath(j,"path")`, `json_array_to_mv()`	JSON
`random()`, `tostring(_time)`, `typeof(x)`	utilities

Hunting patterns

1. Rare parent/child pairs, fleet-wide, via the Endpoint data model (fast)

```
| tstats summariesonly=t count dc(Processes.dest) as hosts min(_time) as first values(Processes.process) as sample
    from datamodel=Endpoint.Processes where earliest=-30d
    by Processes.parent_process_name Processes.process_name
| rename Processes.* as *
| where hosts<=2
| sort hosts count
```

2. First-seen process per host today versus a 30-day baseline

```
| tstats summariesonly=t earliest(_time) as first latest(_time) as last count
    from datamodel=Endpoint.Processes where earliest=-30d by Processes.dest Processes.process_name
| rename Processes.* as *
| where first > relative_time(now(), "-1d@d")
| eval first=strftime(first, "%F %T")
```

3. Long or encoded command lines with a scripting parent

```
index=sysmon EventCode=1 earliest=-7d
| eval cmdlen=len(CommandLine)
| rex field=CommandLine "(?i)\s-(?<flag>e|ec|enc|encodedcommand)\s+(?<b64>[A-Za-z0-9+/=]{20,})"
| where cmdlen>1000 OR isnotnull(b64)
| table _time host User ParentImage Image cmdlen flag b64 CommandLine
```

4. LOLBin stacking: who runs them, from where

```
index=sysmon EventCode=1 earliest=-7d
    (Image="*\\rundll32.exe" OR Image="*\\regsvr32.exe" OR Image="*\\mshta.exe" OR Image="*\\certutil.exe" OR Image
="*\\bitsadmin.exe" OR Image="*\\msiexec.exe" OR Image="*\\wmic.exe" OR Image="*\\cscript.exe" OR Image="*\\wscript.
exe")
| eval image=lower(mvindex(split(Image,"\\"),-1)), parent=lower(mvindex(split(ParentImage,"\\"),-1))
| stats count dc(host) as hosts values(CommandLine) as cmds by image parent
| where hosts<3
```

5. Beaconing: regular intervals and low jitter to the same destination

```
index=zeek sourcetype=zeek:conn earliest=-24h NOT (id.resp_h=10.0.0.0/8 OR id.resp_h=192.168.0.0/16 OR id.resp_h=17
2.16.0.0/12)
| rename id.orig_h as src id.resp_h as dest id.resp_p as dport
| sort 0 src dest _time
| streamstats current=f last(_time) as prev by src dest dport
| eval delta=_time-prev
| where isnotnull(delta)
| stats count avg(delta) as avg_gap stdev(delta) as sd sum(orig_bytes) as out by src dest dport
| eval cv=sd/avg_gap
| where count>=20 AND avg_gap>30 AND cv<0.2
| sort - count
```

6. DNS: long subdomains and entropy with URL Toolbox

```
index=dns sourcetype=zeek:dns earliest=-24h (qtype_name=TXT OR qtype_name=A OR qtype_name=NULL)
| eval labels=mvcount(split(query,".")), qlen=len(query)
| lookup ut_shannon_lookup word as query
| where qlen>60 OR labels>5 OR ut_shannon>4
| rex field=query "(?<domain>[^.]+\.[^.]+)$"
| stats count dc(query) as uniq avg(ut_shannon) as entropy values(query) as sample by id.orig_h domain
| where uniq>50
| sort - uniq
```

7. Rare external destinations from servers (first seen today)

```
| tstats summariesonly=t count earliest(_time) as first from datamodel=Network_Traffic
    where earliest=-30d All_Traffic.src_category=server NOT All_Traffic.dest_ip=10.0.0.0/8 NOT All_Traffic.dest_ip=1
72.16.0.0/12 NOT All_Traffic.dest_ip=192.168.0.0/16
    by All_Traffic.src All_Traffic.dest_ip All_Traffic.dest_port
| rename All_Traffic.* as *
| where first > relative_time(now(), "-1d@d")
```

8. Logon type 3 fan-out: one account to many hosts in an hour

```
index=wineventlog EventCode=4624 Logon_Type=3 earliest=-24h NOT Account_Name="*$" NOT Account_Name="ANONYMOUS LOGON"
| bin _time span=1h
| stats dc(host) as hosts values(host) as host_list by Account_Name _time
| where hosts>=10
```

9. New local administrators

```
index=wineventlog EventCode=4732 earliest=-7d Group_Name=Administrators
| table _time host Subject_Account_Name Member_Name Member_Security_ID
```

10. LSASS access with credential-dump masks (Sysmon 10)

```
index=sysmon EventCode=10 TargetImage="*\\lsass.exe" earliest=-7d
    (GrantedAccess=0x1010 OR GrantedAccess=0x1410 OR GrantedAccess=0x1438 OR GrantedAccess=0x143A OR GrantedAccess=0
x1FFFFF)
    NOT SourceImage IN ("C:\\Windows\\System32\\*", "C:\\Program Files\\*", "C:\\Program Files (x86)\\*")
| stats count dc(host) as hosts values(CallTrace) as calltrace by SourceImage GrantedAccess
| sort hosts
```

11. Run-key stacking across the fleet (Sysmon 13)

```
index=sysmon EventCode=13 earliest=-30d
    (TargetObject="*\\CurrentVersion\\Run\\*" OR TargetObject="*\\CurrentVersion\\RunOnce\\*" OR TargetObject="*\\Wi
nlogon\\*" OR TargetObject="*\\Image File Execution Options\\*")
| rex field=TargetObject "(?<value_name>[^\\\\]+)$"
| stats dc(host) as hosts min(_time) as first values(Image) as writer by value_name Details
| where hosts<=3
```

12. Risky sign-ins: new country for the user, then MFA failures

```
index=azuread sourcetype="azure:aad:signin" earliest=-24h
| rename properties.userPrincipalName as user, properties.ipAddress as src_ip, properties.location.countryOrRegion a
s country, properties.status.errorCode as result
| stats count sum(eval(if(result IN (500121,50074,50076),1,0))) as mfa_failed sum(eval(if(result=0,1,0))) as success
by user country src_ip
| where mfa_failed>=3 OR success>0
| lookup known_user_countries.csv user country OUTPUTNEW seen
| where isnull(seen)
```

13. Office 365 inbox rules that hide or forward mail

```
index=o365 sourcetype="o365:management:activity" Operation IN ("New-InboxRule","Set-InboxRule","Set-Mailbox") earlie
st=-7d
| spath path=Parameters{} output=params
| mvexpand params
| spath input=params path=Name output=pname
| spath input=params path=Value output=pvalue
| where pname IN ("DeleteMessage","MoveToFolder","ForwardTo","RedirectTo","ForwardingSmtpAddress","MarkAsRead")
| stats values(eval(pname.":".pvalue)) as settings by _time UserId ClientIP Operation
```

14. Process to network join without 'join': stitch by ProcessGuid

```
index=sysmon (EventCode=1 OR EventCode=3) earliest=-24h
| eval dest=if(EventCode=3, DestinationIp.":".DestinationPort, null())
| stats values(dest) as dests dc(dest) as n_dest values(CommandLine) as cmd values(ParentImage) as parent by host Pr
ocessGuid Image
| where n_dest>0
| sort - n_dest
```

tstats and data models

```
| tstats summariesonly=t allow_old_summaries=t count from datamodel=Endpoint.Processes
    where earliest=-24h Processes.process_name=powershell.exe by Processes.dest Processes.user _time span=1h
| rename Processes.* as *
```

Data model.object	Key fields	Covers
`Endpoint.Processes`	`dest user process process_name parent_process_name process_path process_guid process_hash`	4688, Sysmon 1, EDR
`Endpoint.Filesystem`	`dest file_path file_name file_hash action`	Sysmon 11/23, EDR
`Endpoint.Registry`	`dest registry_path registry_value_name registry_value_data`	Sysmon 12/13
`Endpoint.Ports`, `Endpoint.Services`	listening ports, services	7045, Sysmon
`Authentication`	`user src dest action app signature_id`	4624/4625, SSH, cloud sign-ins
`Network_Traffic.All_Traffic`	`src dest dest_port bytes_in bytes_out transport action`	firewall, Zeek conn, NetFlow
`Network_Resolution.DNS`	`src query answer record_type`	DNS
`Web.Web`	`src dest url http_user_agent status bytes`	proxy, IIS
`Email.All_Email`	`src_user recipient subject file_name`	mail gateway
`Change.All_Changes`	`user object action dest`	account and config changes

- `summariesonly=t` reads only the accelerated summary: fast, but events arriving late or outside the acceleration window are missing; drop it to include raw events (slow).
- `nodename=` selects an object (`where nodename=Endpoint.Processes`); `prestats=t` feeds a following `stats`/`timechart`.
- `tstats count where index=* by index sourcetype` works on indexed fields with no data model at all: the fastest inventory query there is.
- Data models need CIM-compliant add-ons (Windows TA, Sysmon TA, Zeek TA, Cloud Services add-on) and acceleration turned on per model.

Enterprise Security notes

- **Risk-based alerting**: detections write to `index=risk` with `risk_object`, `risk_object_type`, `risk_score`, `risk_message`, and `annotations.mitre_attack`; a notable fires when a risk object's score exceeds a threshold or spans several tactics in 24 h. Hunts can feed it: `| eval risk_score=20, risk_object=user, risk_object_type="user" | collect index=risk`.
- Notables live in `index=notable`; `| search \` notable`with the macro adds status and owner.` | rest /services/saved/searches | search action.correlationsearch.enabled=1` lists correlation searches.
- Asset and identity lookups (`asset_lookup_by_str`, `identity_lookup_expanded`) add `priority`, `category`, `bunit`; filter hunts by `priority=critical`.
- ESCU (Splunk Security Content) ships hundreds of `tstats` detections mapped to ATT&CK; read them for field names before writing your own, and remember most need `summariesonly=t`.

Performance tips

- Time range first, then `index=` and `sourcetype=`, then indexed terms (`TERM()`), then everything else.
- Use `tstats` for counts and stacking; fall back to raw search only to read the events you shortlisted.
- `fields` (not `table`) early to cut what streams between indexers and the search head; `| fields - _raw` when you do not need the text.
- Avoid `join`, `transaction`, `append` with subsearches, and `map` on big sets; use `stats ... by key`, `lookup`, `eventstats`.
- Sort inside `dedup` (`sortby`) or sort after reducing; `sort 0` removes the 10,000-row default cap.
- Check the **Job Inspector** (`search.log`, "command.search" vs "dispatch.evaluate") to see where time goes; `| noop` with `search_optimization=false` to compare.
- Accelerate data models and use summary indexes for 30-day baselines rather than re-scanning raw data every hunt.
- Leading wildcards (`*shell.exe`) defeat the index; `TERM(powershell.exe)` or `process_name=powershell.exe` on an accelerated model is fast.

Common Windows field names

Concept	`XmlWinEventLog` (Windows TA, modern)	Classic `WinEventLog`	Sysmon TA	CIM
Event id	`EventCode`	`EventCode`	`EventCode`	`signature_id`
Account	`TargetUserName`, `SubjectUserName`	`Account_Name` (multivalue: subject and target)	`User`	`user`, `src_user`
Logon type	`LogonType`	`Logon_Type`		`action`, `authentication_method`
Source IP	`IpAddress`	`Source_Network_Address`	`SourceIp`	`src`, `src_ip`
Process	`NewProcessName`, `CommandLine`	`New_Process_Name`, `Process_Command_Line`	`Image`, `CommandLine`	`process_name`, `process`
Parent	`ParentProcessName`	`Creator_Process_Name`	`ParentImage`, `ParentCommandLine`	`parent_process_name`
Host	`Computer`, `host`	`ComputerName`, `host`	`Computer`, `host`	`dest`
Service	`ServiceName`, `ImagePath` (7045)	`Service_Name`, `Service_File_Name`		`service`, `service_path`
Share	`ShareName`, `RelativeTargetName`, `AccessMask`	`Share_Name`, `Relative_Target_Name`		`file_path`, `action`

Classic `Account_Name` is multivalue (subject first, target second): `mvindex(Account_Name, 1)` for the target. Render the same hunt against the data model whenever you can; the CIM names are stable across add-on versions.

Related: query index (every pattern in KQL and SPL side by side), playbooks, KQL cheat sheet, Windows event IDs, Sysmon.

HUNT SAGE iamjammie/hunt-sage